

Lập trình Java Java script

GV Nguyễn Đức Kiên



1 Javascript

1. Giới thiệu
2. Cách sử dụng
3. Các khái niệm cơ bản
4. Tương tác với DOM

1.1 Giới thiệu

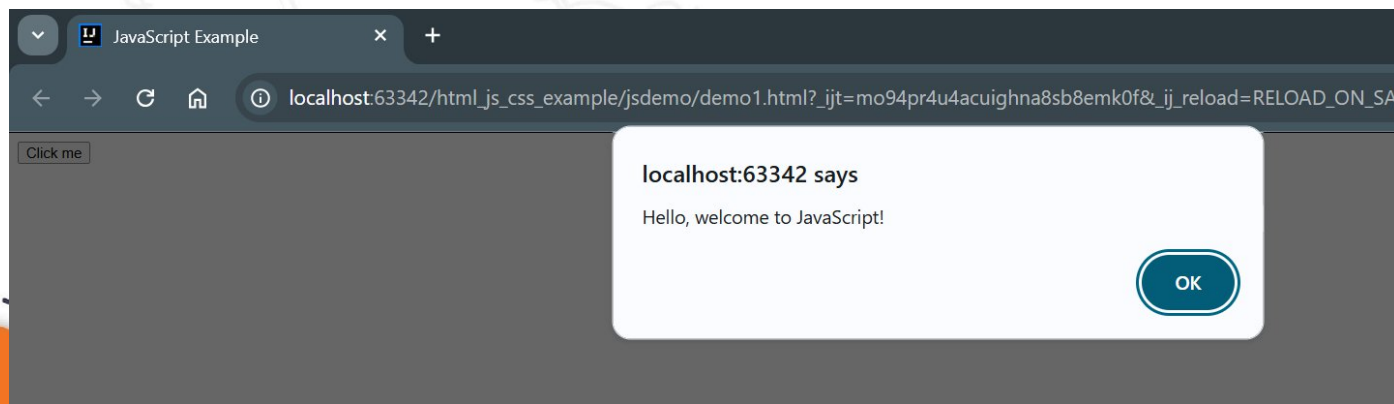
- JavaScript (JS) là một ngôn ngữ lập trình động, thường được sử dụng để phát triển các trang web và ứng dụng web. JavaScript chủ yếu được sử dụng để tạo các tương tác và chức năng động trên trang web, giúp trang web trở nên sinh động và tương tác với người dùng theo cách tự nhiên hơn.
- Các đặc điểm chính của JavaScript:
 - Lập trình phía client:** JavaScript thường chạy trực tiếp trên trình duyệt của người dùng, giúp tạo các tương tác mà không cần phải gửi yêu cầu đến máy chủ.
 - Tương tác người dùng:** JavaScript có thể xử lý các sự kiện người dùng như nhấp chuột, di chuột, nhập liệu và thay đổi nội dung của trang mà không cần tải lại toàn bộ trang.
 - Khả năng thay đổi nội dung động:** JavaScript có thể thay đổi cấu trúc của trang web (DOM) mà không cần phải tải lại trang (còn gọi là AJAX).
 - Đề tích hợp với HTML và CSS:** JavaScript có thể làm việc chặt chẽ với HTML để thay đổi nội dung và với CSS để điều chỉnh giao diện theo yêu cầu.

1.1 Giới thiệu

Ví dụ về JavaScript:

- Trong ví dụ:
 - `<button>`: Tạo một nút mà người dùng có thể nhấn.
 - `onclick="displayMessage()"`: Khi người dùng nhấn vào nút, hàm `displayMessage()` trong JavaScript được gọi, hiển thị một hộp thông báo.
 - `alert()`: Là một phương thức trong JavaScript để hiển thị thông báo cho người dùng.

```
<> jsdemo\demo1.html x
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>JavaScript Example</title>
7  </head>
8  <body>
9      <button onclick="displayMessage()">Click me</button>
10     <script>
11         function displayMessage() { Show usages
12             alert("Hello, welcome to JavaScript!");
13         }
14     </script>
15 </body>
16 </html>
17
```



2 Cách sử dụng

- Để sử dụng JavaScript trong HTML, bạn có thể tích hợp mã JavaScript vào các phần tử HTML theo ba cách chính: inline, internal và external.
- Các cách sử dụng:
 1. Inline JavaScript
 2. Internal JavaScript
 3. External JavaScript

2 Cách sử dụng

- **Inline JavaScript:** Đây là cách chèn trực tiếp mã JavaScript vào một thuộc tính của thẻ HTML, như onclick, onmouseover, v.v. Trong trường hợp này, mã JavaScript sẽ được thực thi khi người dùng tương tác với phần tử đó.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Inline JavaScript Example</title>
7 </head>
8 <body>
9   <button onclick="alert('Hello from JavaScript!')">Click me</button>
10 </body>
11 </html>
```

- Trong ví dụ này, khi người dùng nhấn vào nút, hàm alert() sẽ được gọi và hiển thị một thông báo.

2 Cách sử dụng

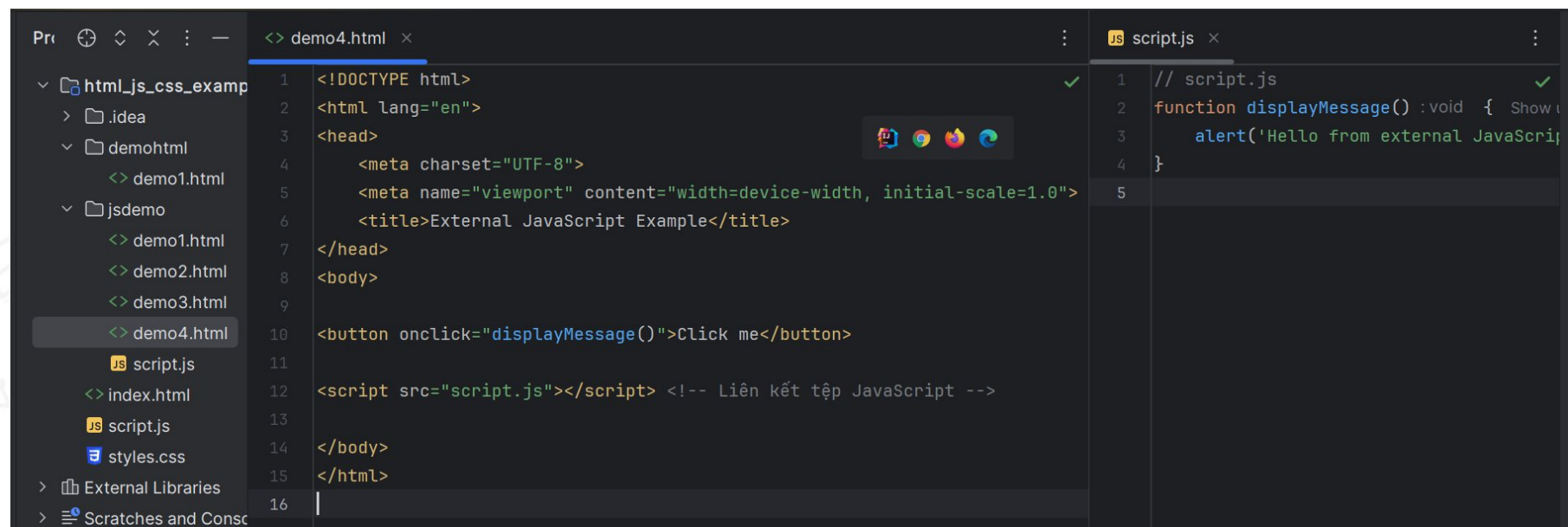
- **Internal JavaScript:** có thể viết mã JavaScript trực tiếp trong tệp HTML bằng cách đặt nó trong thẻ `<script>` trong phần `<head>` hoặc `<body>` của trang HTML.
- Trong ví dụ này, mã JavaScript được đặt trong thẻ `<script>` trong phần `<head>`, và khi người dùng nhấn vào nút, hàm `displayMessage()` sẽ được gọi để hiển thị thông báo.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internal JavaScript Example</title>
  <script>
    function displayMessage() { Show usages
      alert('Hello from internal JavaScript!');
    }
  </script>
</head>
<body>
  <button onclick="displayMessage()">Click me</button>
</body>
</html>
```


2 Cách sử dụng

- **External JavaScript:** Khi mã JavaScript trở nên dài hoặc phức tạp, bạn có thể tách nó ra thành một tệp riêng biệt và liên kết nó với tệp HTML thông qua thẻ `<script>` với thuộc tính `src`. Cách này giúp mã của bạn dễ quản lý hơn, đặc biệt khi phát triển các dự án lớn.

- Trong ví dụ, mã JavaScript được đặt trong tệp `script.js`, và tệp này được liên kết với trang HTML qua thẻ `<script src="script.js"></script>`.



```
<!-- demo4.html -->
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>External JavaScript Example</title>
7 </head>
8 <body>
9
10  <button onclick="displayMessage()">Click me</button>
11
12  <script src="script.js"></script> <!-- Liên kết tệp JavaScript -->
13
14 </body>
15 </html>
16

-- script.js --
1 // script.js
2 function displayMessage() :void { Show
3   alert('Hello from external JavaScript
4 }
5
```


2 Cách sử dụng

- Vị trí đặt thẻ `<script>`:
 - ❑ **Trong thẻ `<head>`:** Nếu muốn JavaScript được tải khi trang HTML được tải, nhưng không làm ảnh hưởng đến tốc độ hiển thị của nội dung, bạn có thể đặt thẻ `<script>` trong phần `<head>`. Tuy nhiên, điều này có thể khiến trang web hiển thị chậm hơn, vì trình duyệt sẽ phải tải và thực thi JavaScript trước khi hiển thị nội dung trang.
 - ❑ **Cuối thẻ `<body>`:** Cách tốt hơn là đặt thẻ `<script>` trước thẻ `</body>` để mã JavaScript chỉ được tải sau khi nội dung trang đã được tải xong. Điều này giúp trang web hiển thị nhanh hơn.

3 Các khái niệm cơ bản

1. Biến (Variables)
2. Kiểu dữ liệu (Data Types)
3. Toán tử (Operators)
4. Câu lệnh điều kiện (if, else, switch)
5. Vòng lặp (Loops)
6. Hàm (Functions)
7. Mảng (Arrays)
8. Đối tượng (Objects)

3.1 Biến (Variables)

- Biến trong JavaScript được sử dụng để lưu trữ dữ liệu mà có thể thay đổi trong quá trình chạy chương trình.
- Các loại biến:
 - ❑ **var:** Khai báo biến, có phạm vi toàn cục hoặc trong hàm.
 - ❑ **let:** Khai báo biến, có phạm vi trong khối mã (block scope), khuyến khích sử dụng thay vì var.
 - ❑ **const:** Khai báo hằng số (biến không thay đổi giá trị), có phạm vi trong khối mã (block scope).

```
JS script.js x
1  let name : string = "John";
2  const age : number = 30;
3  var check : boolean = true;
4
```


3.2. Kiểu dữ liệu (Data Types)

- JavaScript hỗ trợ một số kiểu dữ liệu cơ bản, bao gồm:
 1. String: Chuỗi văn bản.
 2. Number: Số (bao gồm số nguyên và số thập phân).
 3. Boolean: Kiểu dữ liệu logic (true hoặc false).
 4. Object: Đối tượng, dùng để lưu trữ nhiều giá trị dưới dạng khóa-giá trị (key-value pairs).
 5. Array: Mảng, dùng để lưu trữ một danh sách các giá trị.
 6. Null: Đại diện cho giá trị "null", có nghĩa là không có giá trị nào.
 7. Undefined: Đại diện cho một biến chưa được gán giá trị.
 8. Symbol (ES6): Được dùng để tạo các giá trị không trùng lặp.

3.2. Kiểu dữ liệu (Data Types)

- Code ví dụ:

```
JS script.js ×  
1 let name : string = "John";  
2 const age : number = 30;  
3 var check : boolean = true;  
4 let isActive : boolean = true; // Boolean  
5 let person : {age: number, name: string} = { // Object  
6     name: "Alice",  
7     age: 25  
8 };  
9 let numbers : number[] = [1, 2, 3]; // Array  
10
```

3.3. Toán tử (Operators)

Toán tử là các ký hiệu dùng để thực hiện các phép toán hoặc các thao tác trên dữ liệu.

- Toán tử số học: +, -, *, /, % (chia lấy dư), ++ (tăng), -- (giảm).
- Toán tử so sánh: ==, === (so sánh giá trị và kiểu dữ liệu), !=, !==, >, <, >=, <=.
- Toán tử logic: && (và), || (hoặc), ! (phủ định).
- Toán tử gán: =, +=, -=, *=, /=. Toán tử điều kiện (ternary operator): condition ? expr1 : expr2.

```
let x : number = 10;  
let y : number = 5;  
let result : number = x + y; // Toán tử số học  
let isEqual : boolean = (x == y); // Toán tử so sánh  
let isValid : boolean = (x > y && y > 0); // Toán tử logic
```

3.4. Câu lệnh điều kiện (if, else, switch)

Sử dụng để kiểm tra điều kiện và thực hiện hành động tương ứng.

- **if:** Kiểm tra một điều kiện và thực hiện hành động nếu điều kiện đúng.
- **else:** Thực hiện hành động nếu điều kiện if sai.
- **switch:** Kiểm tra một giá trị với nhiều trường hợp khác nhau.

```
if (age > 18) {  
    console.log("Adult");  
} else {  
    console.log("Child");  
}  
  
let color : string = "red";  
switch (color) {  
    case "red":  
        console.log("Red color");  
        break;  
    case "blue":  
        console.log("Blue color");  
        break;  
    default:  
        console.log("Other color");  
}
```


3.5. Vòng lặp (Loops)

JavaScript cung cấp nhiều cách để lặp qua các phần tử hoặc thực hiện hành động nhiều lần.

- **for:** Dùng để lặp qua một số lần xác định.
- **while:** Lặp khi điều kiện là true.
- **do...while:** Lặp ít nhất một lần và tiếp tục nếu điều kiện đúng.
- **for...in:** Dùng để lặp qua các thuộc tính của đối tượng.
- **for...of:** Dùng để lặp qua các phần tử trong một mảng.

```
// for loop
✓ for (let i : number = 0; i < 5; i++) {
    console.log(i); // In ra 0, 1, 2, 3, 4
}

// while loop
let count : number = 0;
✓ while (count < 5) {
    console.log(count);
    count++;
}

// for...of loop (dùng với mảng)
let numbers : number[] = [1, 2, 3, 4];
✓ for (let number : number of numbers) {
    console.log(number);
}
```

3.6. Hàm (Functions)

Hàm trong JavaScript là một khối mã có thể tái sử dụng, giúp tổ chức mã và thực hiện các tác vụ.

- ❑ **Khởi tạo hàm:** Sử dụng từ khóa function để khai báo một hàm.
- ❑ **Hàm không có tham số:** Hàm không nhận bất kỳ tham số nào.
- ❑ **Hàm có tham số:** Hàm có thể nhận tham số vào khi gọi.
- ❑ **Hàm có giá trị trả về:** Hàm có thể trả về một giá trị.

```
// Hàm không có tham số
function greet() :void { Show usages
    console.log("Hello, World!");
}

// Hàm có tham số
function add(a, b) { Show usages
    return a + b;
}

greet(); // Gọi hàm greet
// Gọi hàm add và in ra kết quả 5
console.log(add(a: 2, b: 3));
```

3.7. Mảng (Arrays)

- Mảng trong JavaScript là một đối tượng dùng để lưu trữ nhiều giá trị theo thứ tự. Mảng có thể chứa bất kỳ kiểu dữ liệu nào.

```
let fruits : string[] = ["apple", "banana", "cherry"];  
console.log(fruits[0]); // In ra "apple"  
fruits.push("orange"); // Thêm phần tử "orange" vào cuối mảng
```


3.8. Đối tượng (Objects)

- Đối tượng trong JavaScript là một kiểu dữ liệu phức tạp, dùng để lưu trữ dữ liệu dưới dạng cặp khóa-giá trị (key-value pairs).

```
let person : {...} = {  
  name: "John",  
  age: 30,  
  greet: function() : void {  
    console.log("Hello, " + this.name);  
  }  
};  
  
console.log(person.name); // In ra "John"  
person.greet();           // Gọi phương thức greet() trong đối tượng
```